Turing Machine

A computing device with finite state control, an unbounded tape as memory, a tape head which scans a given cell on tape.
Initialized in starting state q_0, input string $\omega = \omega_1\omega_2 \dots \omega_n \in \Sigma^*$ on tape, head points to ω_1 & all othe tape cells intialized to blank symbol B.

Def

A Deterministic Turing Machine (DTM) is given by a 7-tuple

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej}), \text{ where}$$

Q is a finite set of states

Σ is a finite alphabet for input string ω

$\Gamma = \Sigma \cup \{B\} \cup \{\text{any additional symbols needed by TM}\}$
e.g., string delimiters etc.
consider this alphabet for the tape
Σ does not contain B.

$q_0 \in Q$ starting state

$q_{acc} \in Q$ accept state

$q_{rej} \in Q$ reject state

} Acceptance & Rejection is instantaneous i.e., cannot transition out of these states
Note $q_{acc} \neq q_{rej}$

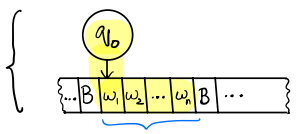
Transition function $\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, S\}$
- Q: current state
- Γ: current symbol on tape pointed to by head
- Q: new state
- Γ: new symbol to write on tape
- $\{L, R, S\}$: Move head Left, Right or Stay only one cell

Configuration of TMs

Consider them as snapshot of TMs at some instant during computation.

Initial configuration (start of computation)

can be written as just $q_0\omega$



In general, Configuration C_k yields $\vdash$ configuration C_ℓ, i.e.,

$u\ a\ q_i\ b\ v$ yields $u\ q_j\ a\ c\ v$ $\quad u, v \in \Gamma^*$, $a, b, c \in \Gamma$

if there exists the following entry for δ:

$$\delta(q_i, b) = (q_j, c, L)$$

transition to q_j, overwrite input b with c & move head left one cell

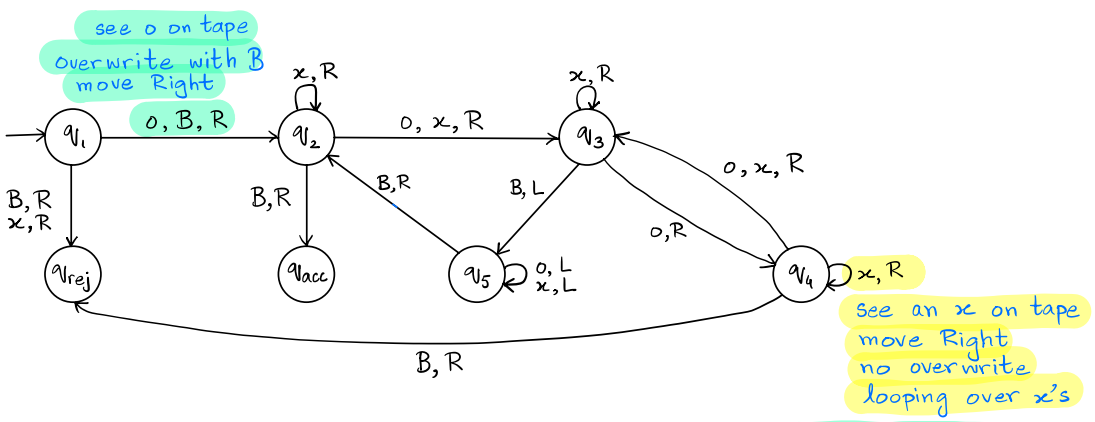
A TM M accepts an input string w, if there exist a sequence of configurations, $C_1, C_2, \ldots, C_m$, where

1. C_1 is initial configuration
2. Each C_i yields C_{i+1} for $i \in \{1, 2, \ldots, m-1\}$
3. C_m is an accepting configuration
   ↳ has accepting state, ie., q_{acc}

Similar definition for rejecting configuration

$L(M)$ — Language recognized by M is the set of strings M accepts.

e.g., What is the language of following M? $\Sigma = \{0\}$, $\Gamma = \{0, x, B\}$



Recommend running the TM above on multiple inputs (use JFLAP on LMS) to establish its language is:

$L = \{ 0^{2^n} \text{ for } n \geq 0 \}$, i.e., string of 0's with length powers of 2.

We can also specify TM solutions at different levels of detail & abstraction. Above is a full transition diagram.

Implementation Level Description

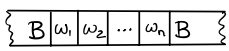
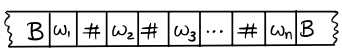
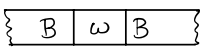
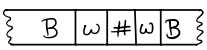
On input w:

0. Overwrite the first 0 with a B & accept if input ends.
1. Sweep left to right across input, marking with an x every other 0.
2. If step 1 contained a single 0, accept.
3. If step 1 contained more than 1 marked 0 & the number of 0's marked were odd, reject.
4. Return head to start of input.
5. Go to step 1.

High Level

Accept if only a single 0.
Keep halving the number of 0's in input.
Accept if left with only a single 0, o/w reject if left with odd number of 0's at any stage.

Programming Primitives for TM Design

1. Move left or right till first B is found.
2. Shift string w one cell to the left or right.
3. Convert  into 
4. Simulate arbitrary alphabet with $\Sigma = \{0,1\}$.
5. Mark symbol 'a' on tape with $\overline{a}$, i.e., remember location.
6. Copy string w on tape, i.e.,  into 
7. Increment / decrement in binary
8. Implement basic string / arithmetic operations
9. Simulate RAM

⋮

Simulate a TM (Will see next week).

Variants of TMs

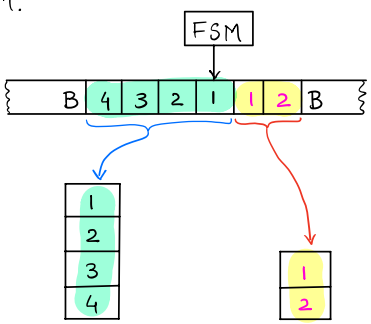
1. A TM with k-tapes & heads can be simulated by a standard TM.
(Imagine a parallel computation being simulated by a single processor).

Key Idea Partition the single tape into k sections that can be expanded as needed. Simulate a single transition of the i^{th} tape on the i^{th} section. Iterate.

2. A machine with two stacks instead of a single unbounded tape is equivalent to a TM.

Key Idea

Manage transitions on tape by pushing & popping from stacks as needed.



Stack 1
everything to the left on tape including current position

Stack 2
everything to the right on tape

3. Non-Deterministic TMs (NTMs)

An NTM can be simulated by a DTM.

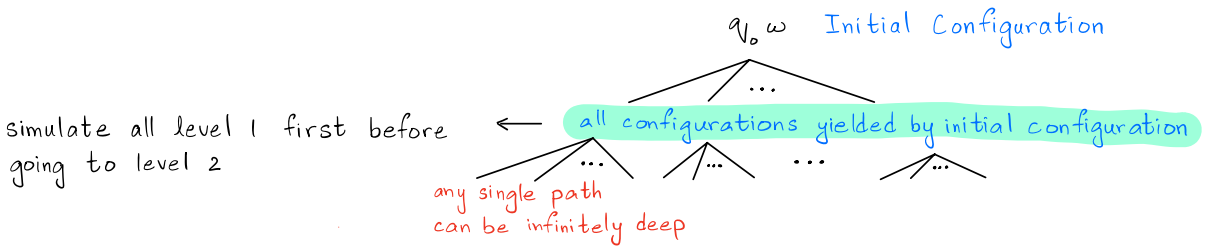
Transition function for NTM

$$\delta : Q \times \Gamma \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R, S\})$$

↳ Powerset

Key Idea

Do a BFS search of the computation history tree.
Make sure you understand why DFS won't work.



simulate all level 1 first before going to level 2

Decidability

Since on input w, a TM M has three possibilities, i.e.,

1. accept
2. reject
3. go in an infinite loop

we have to be careful about defining a language of a TM.

Turing Recognizable (TR) / Recursively Enumerable

A language L is TR if there exists a TM M such that for all strings $w \in L$, TM M halts & accepts. (May loop or reject other strings)

co-Turing Recognizable (TR) / co-Recursively Enumerable

A language L is co-TR if there exists a TM M such that for all strings $w \notin L$, TM M halts & rejects. (May loop or accept other strings)

Decidable / Recursive

A language is decidable if it is both TR & co-TR.